

# Integrating agentic risk signalling in trusted research environments: Automating VEX with Agent2Agent protocols and model context protocol (MCP) in SACRO and TRevolution pipelines

Petar Radanliev<sup>a,c,e,\*</sup>, Kayvan Atefi<sup>e</sup>, Omar Santos<sup>b</sup>, Carsten Maple<sup>c,d</sup>

<sup>a</sup> Department of Computer Sciences, University of Oxford, Wolfson Building, Parks Rd, Oxford OX1 3QG, UK

<sup>b</sup> Cisco Systems, RTP, NC, United States

<sup>c</sup> The Alan Turing Institute, British Library, 96 Euston Rd., London NW1 2DB, UK

<sup>d</sup> University of Warwick-WMG, 6 Lord Bhattacharyya Way, Coventry CV4 7AL, UK

<sup>e</sup> University of the West of England, Bristol BS16 1QY, UK

## ARTICLE INFO

### Keywords:

Vulnerability-Exploitability eXchange (VEX)  
Trusted Research Environments (TREs)  
Agent2Agent Protocol (A2A)  
Model Context Protocol (MCP)  
Federated AI governance  
Automated vulnerability signalling  
Reproducible research pipelines  
Secure metadata propagation  
AI risk management

## ABSTRACT

This study presents a framework for automating the generation and validation of machine-readable vulnerability statements, known as Vulnerability-Exploitability Exchange (VEX) artefacts, within secure research environments. The work addresses a critical limitation in existing vulnerability reporting, where static scoring systems often fail to capture whether a flaw is truly exploitable in a specific analytic context. By integrating structured metadata capture, runtime instrumentation, and cryptographically verifiable provenance, the framework classifies vulnerabilities as affected, fixed, or not relevant, supported by machine-readable evidence bundles. The methodology was evaluated using containerised applications seeded with deliberately vulnerable components. Software bill of materials and vulnerability scanners were applied to generate baseline inventories, while reproducibility frameworks validated that results could be independently replicated. Findings demonstrate that automated VEX generation can reduce false positives by distinguishing theoretical from actionable risks, thereby improving security assurance and reproducibility in federated infrastructures. At the same time, the research acknowledges significant challenges. Computational overhead from multi-layered monitoring, dependence on external tools, and the risk of false negatives introduce barriers to adoption. Broader pilot studies across heterogeneous domains and benchmarking on standardised testbeds are required to enhance generalisability. Privacy concerns from extensive runtime monitoring and the need for sustainable maintenance models also demand attention. By combining automation with human oversight and aligning with emerging standards, the study contributes a reproducible, auditable, and context-sensitive approach to vulnerability management. The work provides both a proof-of-concept and a roadmap for refining security practices in sensitive computational environments.

## 1. Introduction

This study presents a system for generating Software Bills of Materials (SBOMs) [1,2] for securing software [3] supply chain insecurity [4], and managing machine-readable Vulnerability-Exploitability eXchange (VEX) [5,6] artefacts within Trusted Research Environments (TREs) [7] using agentic automation [8]. The implementation builds on the SACRO pipeline for semi-automated research output verification and extends the TRevolution architecture [9] with support for structured

vulnerability reporting. The primary objective is to produce verifiable, digitally signed VEX statements tied to specific analysis jobs, models, and containers executed within TREs.

Unlike SBOMs [10], which provide static inventories of software components, VEX artefacts encode the current exploitability status of known vulnerabilities under specific execution conditions. In TREs, where analytical jobs run in isolated containers and outputs are subject to disclosure control, generating valid VEX statements requires instrumentation of container execution, runtime analysis of software

\* Corresponding author at: Department of Computer Sciences, University of Oxford, Wolfson Building, Parks Rd, Oxford OX1 3QG, UK.

E-mail address: [petar.radanliev@cs.ox.ac.uk](mailto:petar.radanliev@cs.ox.ac.uk) (P. Radanliev).

<sup>1</sup> ORCID: 0000-0001-5629-6857.

dependencies, and alignment with provenance-tracking mechanisms.

To address these requirements, this work integrates two emerging technologies. The Model Context Protocol (MCP) [11] captures structured execution metadata at pre-execution, runtime, and post-execution stages. The Agent2Agent Protocol [12] enables container-resident autonomous agents to exchange messages securely across isolated contexts. Together, these tools support a system in which agents monitor job execution, detect vulnerable components based on observed state, and generate VEX artefacts in compliance with schema specifications. The MCP provides contextual binding between the VEX record and the specific job instance, including hash values for containers, models, and input datasets. These are some of the key problems with expanding SBOMs [13], into Artificial Intelligence BOMs (AIBOM) [14], which is supported by OWASP [15].

The system is implemented as an extension to the SACRO architecture [16], with integration into language-specific toolkits used for statistical disclosure control. Agentic components are deployed as sidecar processes in job containers. These agents access dependency graphs, query CVE databases (e.g., NVD and OSV) [17–19], evaluate the presence and impact of vulnerabilities, and generate digitally signed VEX statements with runtime evidence. Generated artefacts are validated against the SACRO schema extensions and stored alongside output metadata.

The architecture is evaluated using synthetic and real-world data pipelines aligned with the SACRO workflow. The results demonstrate that agentic generation of VEX artefacts can be executed during the job lifecycle with no disruption to existing audit controls. The VEX artefacts are validated for schema compliance and reproducibility across federated TRE instances. This work provides the technical specifications, validation scripts, and schema templates required for deployment in TREs aligned with the SACRO and TREvolution infrastructures.

### 1.1. Background to the cybersecurity problem

In research and development, cybersecurity comes late in the list of operations [20]. Although a security engineer would always advise developers to do their patches and updates, and the response is quite often, is it necessary? It's hard; why should I do this? And this debate is a big problem [21]. One proposed solution for the cybersecurity problem, was replacing the Common Vulnerability Scoring System Calculator (CVSS) [22,23] and the Base Score, the Temporal Score and the Environmental Score [24] with three possible strategies based on the vulnerability score only. These options are (1) if CVSS > 8.9, then patch now; (2) if CVSS > 6.0, patch later; and (3) if CVSS < 7.0, then accept the risk [25]. The first problem with this solution is that we cannot position where the risk is. Second is the scaling problem, where the number of advisories/CVEs on the rise, and the number of patches without CVE entries in the National Vulnerability Database (NVD) is not visible (the term visibility in cybersecurity refers to the capability to see all aspects of the digital footprint as well as the risks and vulnerabilities within it.)

With present methods, PDF and text files are usually shared and not signed. This exposes the operator because if vulnerabilities are shared as PDFs and not signed by the operators, a malicious actor could send fake advisories. Automation also presents problems [26].

## 2. Literature review

The implementation of machine-readable, context-sensitive vulnerability assertions in TREs requires synthesising advances in vulnerability reporting standards, reproducibility frameworks, and secure analytics orchestration. The VEX specification, developed under the stewardship of the U.S. Department of Commerce, was introduced to contextualise known software vulnerabilities (CVEs) within specific environments and execution states [27], thereby reducing false positives and alert fatigue in security operations [5]. These structured advisories build upon SBOM frameworks to declare whether particular vulnerabilities are

exploitable, not present, or have been remediated. While traditional SBOMs catalogue software dependencies [28], they do not address dynamic runtime conditions [29]. VEX mechanisms address this gap [30, 31], especially when linked with vulnerability feeds such as OSV or local NVD mirrors.

Recent efforts in secure analytics within TREs, such as SACRO [16], have adapted VEX to support federated reproducibility, secure audit trails, and real-time vulnerability suppression. SACRO advances reproducibility by embedding model context and execution lineage directly into the output artefact, drawing from principles established In-Toto and Sigstore. The integration of cryptographically verifiable attestations with analytic job metadata aligns with the Model Context Protocol (MCP), which provides verifiable bindings between input data, models, and execution environments [8]. This integration is particularly significant in GRAIMatter-compliant infrastructures, where digital provenance and reproducibility are prioritised for sensitive health and public data research [32].

The design of context-aware vulnerability assessments within TREs further extends traditional scanning workflows. By evaluating component presence and exploitability within constrained execution contexts (e.g., no egress, sandboxed execution, mandatory disclosure controls), researchers have demonstrated substantial noise reduction in vulnerability alerting [33]. These conditions are enforced using policy engines such as Kyverno [34], which ensure declarative enforcement of resource, access, and data movement policies throughout the analytic lifecycle.

Furthermore, the structured output from these systems, formatted using schemas such as CSAF or OpenVEX, has been integrated into audit registries for long-term verification and lineage tracking [21,26,35,36]. This aligns with current guidance from the Cybersecurity and Infrastructure Security Agency (CISA) [37], which emphasises the need for VEX artefacts to be cryptographically signed and verifiable across lifecycle events [27,38].

The broader ecosystem of tools such as Dependency-Track [39] demonstrates operational feasibility in producing and consuming VEX assertions in production CI/CD pipelines. These tools, when augmented by reproducibility-aware systems like SACRO and federated orchestration models developed in TREvolution [40], position VEX as a viable standard for enhancing digital trust in sensitive computational environments. Efforts to improve vulnerability management in software supply chains have increasingly centred on standards designed to enhance transparency, interoperability, and automation. The SBOM, formalised under the U.S. National Telecommunications and Information Administration and codified through initiatives such as SPDX (ISO/IEC 5962:2021), provides structured inventories of software components. SBOMs have been widely adopted in both open-source and regulated domains because they enable downstream consumers to identify dependencies and link them to known vulnerabilities in public databases such as the National Vulnerability Database (NVD) and the Open Source Vulnerability (OSV) schema. While effective for establishing baseline transparency, SBOMs remain static artefacts and do not capture runtime conditions or context-specific mitigations that determine whether a vulnerability is exploitable in practice.

The VEX specification addresses some of these shortcomings by allowing producers to declare whether a vulnerability affects their product in a given context. This distinction reduces false positives and provides greater clarity than an SBOM alone. However, current implementations of VEX, such as OpenVEX and CISA VEX, often depend on manual assertions of exploitability and lack automated links to reproducible runtime evidence. As a result, they improve communication but do not provide independently verifiable proof that the stated exploitability status is correct.

CSAF complements SBOMs and VEX by providing a standardised format for publishing advisories. CSAF enables structured communication of vulnerabilities and mitigations, particularly at scale across organisations. Nevertheless, CSAF advisories typically remain detached

from runtime telemetry or reproducibility pipelines, meaning they cannot demonstrate whether vulnerabilities were observed, triggered, or mitigated under controlled execution.

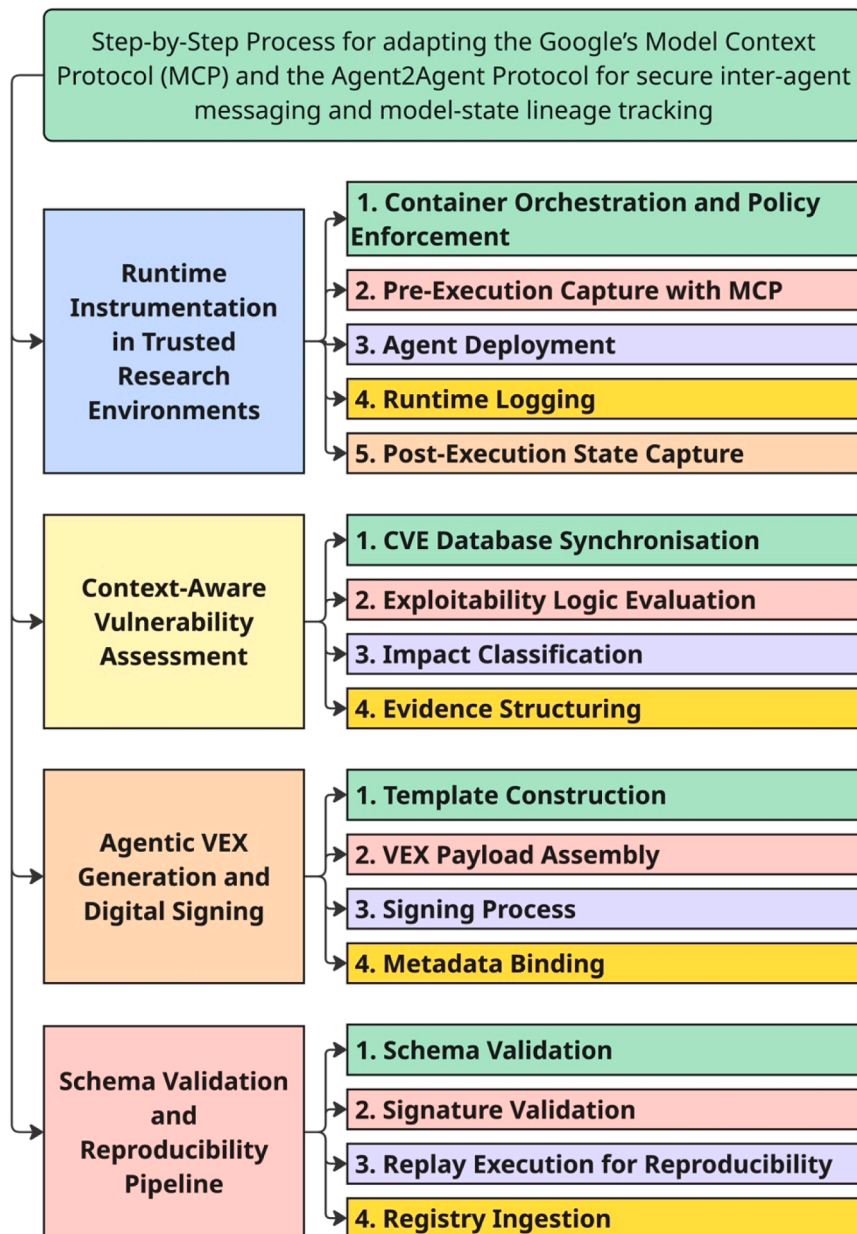
Similarly, the NVD and OSV provide authoritative feeds of vulnerability intelligence but are primarily designed for broad applicability rather than environment-specific risk. They catalogue known issues, assign severity through the Common Vulnerability Scoring System (CVSS), and distribute machine-readable data. Yet, these databases do not incorporate the isolation constraints, privilege restrictions, or execution traces that may render a vulnerability non-exploitable within a given environment.

The framework proposed in this study extends and complements these standards by binding machine-readable advisories to reproducible runtime evidence. By integrating SBOM inventories with live execution

traces, filtering vulnerability data from NVD and OSV through environment-specific constraints, and generating VEX artefacts that are cryptographically signed and schema-validated, the system ensures that exploitability claims are both context-aware and verifiable. In doing so, it bridges the gap between declarative standards such as SBOM, CSAF, and VEX, and operational assurance mechanisms, offering a reproducible pathway to trust in vulnerability management within secure and federated research environments.

### 3. Methodology

The methodology integrates runtime instrumentation, agentic analysis, and protocol-based context embedding into the SACRO framework, and is based on adapting the MCP and the A2A Protocol for secure inter-



**Fig. 1.** Methodology for automated generation, signing, and validation of machine-readable vulnerability exploitability exchange artefacts within secure research environments. The diagram shows how runtime instrumentation, agentic analysis, and context embedding are integrated using the Model Context Protocol and Agent2Agent Protocol to ensure reproducibility and verifiable security attestations.

*Alt-text:* A workflow diagram showing the stages of VEX generation: runtime instrumentation, metadata capture, protocol-based context embedding, agent communication, and validation, resulting in signed vulnerability reports.

agent messaging and model-state lineage tracking. The methodology design (detailed in Fig. 1) is based on the requirement for runtime instrumentation in TREs. The first stage involves deploying agents to observe and log system state during execution of a research job within the SACRO-TRE container architecture. This process ensures that the VEX artefact reflects the static component presence and the actual software execution paths and runtime behaviours. Fig. 1, details the step-by-step process for integrating MCP and A2A for Runtime Instrumentation in TREs.

Fig. 1 presents the technical and procedural methods used to automate the generation, signing, and validation of machine-readable VEX artefacts within TREs, integrating runtime instrumentation, agentic analysis, and protocol-based context embedding into the SACRO framework, applying MCP and the A2A Protocol for secure inter-agent messaging and model-state lineage tracking.

The methodology applied different vulnerability scanners, such as Grype [41], Syft [29], and computational reproducibility tools such as ReproZip [42]. The data was extracted from existing databases on SBOMs [43], and combined with new SBOMs data produced using the SIT 'Accurate, Compliant SBOM Generator with Incremental Construction' [44], GitHub [45], LiPSBOMaker [46], and evaluated with existing SBOM evaluation methods for national critical infrastructure [47], open source tools for SBOM supply chain security [48], and comparative analysis on the 'Claim vs. Capability' [49], research directories on software supply chain security [50], and public provenance repositories [51], to test the scientific model validation [52].

### 3.2. Step-by-step process for container orchestration and policy enforcement

Within the SACRO-aligned TRE, computational workloads are encapsulated as isolated execution units using containerisation technologies such as Docker and Singularity, depending on the security profile and kernel access constraints of the hosting infrastructure. Each research job is submitted as a containerised package, which ensures consistent, reproducible, and auditable execution by capturing the full software environment, dependencies, and control parameters.

The orchestration of these containers is handled via an automated scheduler, Kubernetes in cloud-based deployments or SLURM in high-performance computing (HPC) environments, configured with strict security and resource governance policies. When a job is queued, the orchestration layer enforces declarative resource constraints, which are specified in a structured job manifest. These include:

- Memory allocation limits (`limits.memory` and `requests.memory` in Kubernetes).
- CPU quotas and affinity rules (`limits.cpu`, `topologyKey`, `nodeSelector`).
- Execution time constraints (`activeDeadlineSeconds`, SLURM's `-time` flag).
- Network egress restrictions (via Kubernetes NetworkPolicies or HPC firewalls).
- Mount propagation settings (read-only file systems, write-ahead logging isolation).
- Environment variable whitelisting (to prevent side-channel leakage of secret tokens or credentials).

Each job submission is accompanied by a metadata envelope, which includes a unique job identifier, cryptographic hash of the container image (`sha256:`), and a pointer to the data source under analysis (e.g. dataset UUID or S3 object manifest). Upon submission, a pre-execution policy engine, implemented using OPA (Open Policy Agent) or similar access control mechanisms, evaluates whether the job complies with organisational security policies and data access permissions before

scheduling.

During orchestration, a container runtime security scanner is optionally invoked (e.g., Anchore Engine, Clair) to perform static analysis on the container image for known CVEs prior to execution. This provides an early-stage risk triage aligned with the broader VEX generation framework.

Once admitted, the orchestrator initiates the container instance within a namespace-isolated node pool, enforcing:

- Kernel capability restrictions (via `seccomp`, AppArmor, or Singularity runtime profile).
- Non-persistence of local state (via ephemeral volumes or emptyDir mounts).
- Restricted inter-process communication (IPC) and user namespace isolation (`usersns-remap` or Singularity `-fakeroot`).

These orchestration controls ensure that execution provenance, runtime behaviour, and subsequent VEX assertions are strictly scoped to the declared software boundary, enabling trust in the downstream automation pipeline.

### 3.3. Step-by-step process for pre-execution capture using MCP

In the SACRO-TRE environment, the initiation of any computational job begins with a structured and verifiable capture of the analytic context. This is implemented through the MCP, which formalises the pre-execution phase by generating a tamper-evident, machine-readable record of the analytic state prior to execution. The MCP framework ensures that all subsequent steps, ranging from vulnerability assessment to federated reproducibility, are grounded in a reproducible and cryptographically signed execution context.

Each submitted analytic task is encapsulated in a containerised workload (e.g., Docker or Singularity), with orchestration policies defining compute, memory, and runtime constraints enforced via Kubernetes, SLURM, or equivalent workload managers. Before the container is permitted to execute, an MCP agent initiates the capture process through an orchestrator-integrated pre-execution hook. This agent systematically records the full analytic context across three key dimensions: environment configuration, declared inputs, and software dependency metadata.

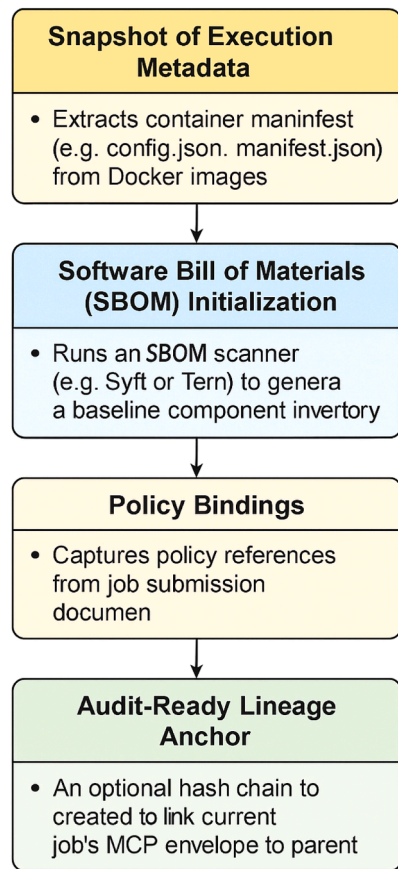
The environment capture begins by extracting runtime metadata including the container image digest, resource allocation parameters, environment variables, container entypoints, input dataset URIs, and declared analytic parameters. These elements are serialised using a canonical JSON schema and hashed using SHA-256 to create a secure, tamper-proof execution fingerprint.

Next, the MCP agent performs a full inventory of software dependencies present in the execution environment. This is achieved using language-specific inspection tools (e.g., Syft, Tern, or custom parsers for R, Python, or Java environments), which extract and record metadata such as package names, versions, provenance URLs, licensing information, and cryptographic checksums. Container layers and virtual environments are also parsed to capture version-locked configurations (`requirements.txt`, `renv.lock`, `environment.yml`, etc.). This dependency inventory serves as the basis for the SACRO-specific AIBOM document generated later in the pipeline.

In parallel, the agent ingests policy artefacts governing the execution environment, including isolation constraints, disclosure control rules, and runtime access permissions. These artefacts are linked to the context record (see Fig. 2) and, where applicable, attested using cryptographic signing infrastructure (e.g., HSM-backed keys managed by the SACRO-TRE's internal security layer or an integrated KMS service).

All collected data is compiled into a structured MCP document, validated against the SACRO-specific JSON schema, and committed to





**Fig. 2.** Pre-execution capture process using the Model Context Protocol. The diagram illustrates how analytic context, including container image, environment parameters, dataset references, and dependency inventories, is serialised into a tamper-evident metadata record that serves as the foundation for reproducible and auditable execution.

*Alt-text:* A schematic showing MCP agents capturing container configuration, inputs, dependencies, and policies before execution, storing them in a structured JSON document with cryptographic hashes.

an immutable provenance ledger maintained within the TRE. This process can be seen in Fig. 2. In workflows where analytic jobs build upon previously executed tasks, the MCP document includes a cryptographic reference to prior context hashes, forming a verifiable execution lineage chain.

This structured pre-execution capture ensures that all analytic jobs are reproducible by design, with full traceability over environment configurations, dependency graphs, and declared inputs. Moreover, it provides the foundation for downstream automation tasks such as vulnerability disclosure via the VEX schema, federated execution tracking, and agentic orchestration using the Agent2Agent protocol stack.

### 3.4. Step-by-step process for agent deployment

To ensure runtime traceability, exploitability detection, and software provenance validation within the SACRO-compliant TRE architecture, a dedicated instrumentation agent is deployed alongside each analytical job. The deployment method is contingent upon the orchestration system in use. In Kubernetes-based TREs, the agent is instantiated as a sidecar container within the same pod as the primary analysis job. This design enables the agent to share network, file system, and process namespaces with the main workload, thereby granting direct observability without altering the containerised job's internal logic. In non-

Kubernetes contexts, such as those using Singularity or SLURM, the agent is configured as an init process by wrapping the job's command-line entrypoint. This ensures the agent launches prior to execution and maintains full visibility throughout the job lifecycle.

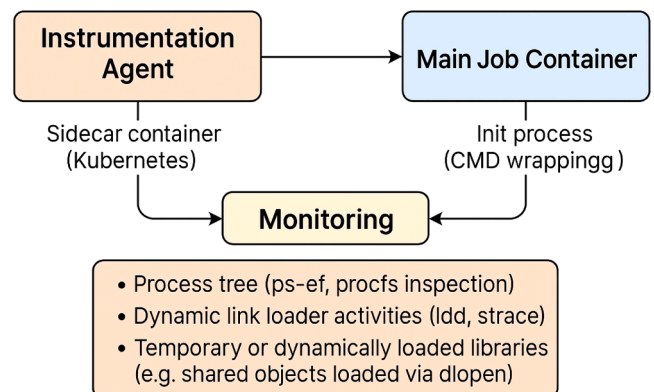
Once deployed, the runtime agent performs continuous monitoring of the execution environment to collect telemetry critical for downstream VEX assertions and reproducibility claims. The agent captures the complete process tree through /proc inspection and process enumeration, enabling the detection of any unauthorised process spawns or deviations from the declared workload. It further inspects dynamic linking behaviour by leveraging tools such as ldd, strace, and auditd to identify runtime-resolved libraries that may not have been declared in the original SBOM. These observations are formalised as SBOM deltas, which are appended to the provenance record. The agent also detects ephemeral artefacts, such as shared objects dynamically loaded via system calls like dlopen, and computes their cryptographic hashes for inclusion in the runtime manifest. For high-assurance deployments, optional support for extended Berkeley Packet Filter (eBPF) tracing allows for deeper analysis of memory events and system call anomalies, providing early detection of exploit behaviours or unexpected binary injections. Fig. 3, helps visualise how the agent deployment is conducted as a runtime instrumentation agent and is deployed as a sidecar container if using Kubernetes, or an init process within the main job container (via CMD wrapping).

Fig. 3, details how the agent monitors process tree (ps -ef, procs inspection), dynamic link loader activities (ldd, strace), and temporary or dynamically loaded libraries (e.g. shared objects loaded via dlopen).

All telemetry generated during this stage is serialised into a structured format and embedded into the job's MCP envelope. This metadata is then cryptographically signed and stored in an immutable ledger or object store, ensuring audit-ready validation of execution lineage. By enforcing such instrumentation across the analytic workflow, the SACRO-TRE architecture establishes a robust chain of trust for runtime environments, supporting post hoc reproducibility, automated exploitability scoring, and agent-to-agent protocol handoffs in federated research infrastructures.

### 3.5. Step-by-step process for runtime logging

To enable traceability and retrospective validation of analytic workflows, the runtime instrumentation agent engages in continuous



**Fig. 3.** Runtime instrumentation agent deployment models for secure research environments. The figure shows deployment either as a sidecar container within Kubernetes pods or as an init process in Singularity/SLURM contexts, enabling continuous monitoring of process trees, libraries, and runtime behaviours.

*Alt-text:* A diagram comparing two deployment methods for monitoring agents: one integrated as a sidecar container in Kubernetes, the other wrapped as an init process in non-Kubernetes systems, both providing runtime visibility.

logging throughout the container's execution lifecycle. This logging process captures transient system and application behaviours that may not be evident during static or pre-execution analysis, particularly in dynamic environments characteristic of TREs.

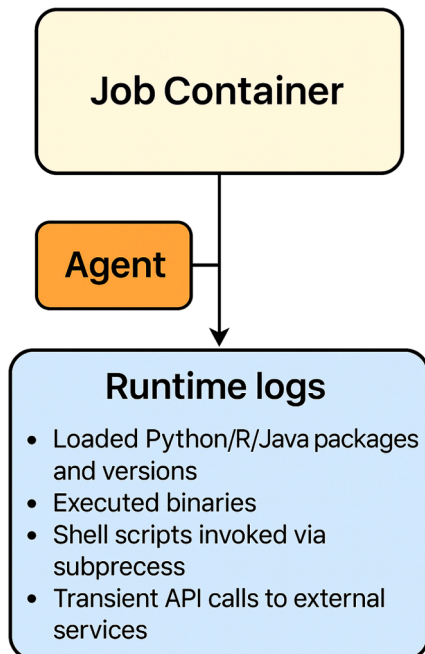
Specifically, the agent records all dynamically loaded language-level dependencies, such as Python modules, R packages, and Java libraries. For Python, this includes the output of `pip list` or `pkg_resources`, while in R, session-specific packages are extracted via `sessionInfo()` or `installed.packages()`. Java classpath dependencies are monitored through runtime instrumentation of class loaders and `.jar` loading events. This ensures that every library invoked at runtime is recorded with its version identifier and source, forming a part of the environment's provenance profile.

In addition to language-level packages, the agent tracks all binaries executed within the containerised session. This includes direct invocations via shell endpoints as well as those launched through subprocess interfaces (e.g., Python's `subprocess.run()` or Bash pipes). Each execution event is logged with command-line arguments, time-stamp, and process ID to establish a verifiable audit trail.

Furthermore, any shell scripts invoked, either as part of automation pipelines or user-provided tools, are captured in their entirety. Their content, invocation parameters, and associated file descriptors are logged for forensic traceability.

Crucially, the agent also monitors outbound API calls or data exchanges initiated by the analytical process, including requests to external services or public endpoints. This is achieved through dynamic tracing of socket activity and HTTP client libraries (e.g., `requests`, `http`, `curl`). These observations are flagged in near-real-time, allowing enforcement of disclosure control and detection of unauthorised network interactions within the TRE execution boundary.

Fig. 4, illustrates the runtime logging activities performed by the



**Fig. 4.** Runtime logging and provenance capture during analytic job execution. The figure illustrates how dynamically loaded packages, executed binaries, shell scripts, and outbound API calls are continuously logged, hashed, and stored in a secure provenance ledger for forensic traceability and reproducibility validation.

*Alt-text:* A technical diagram showing runtime logs collected for binaries, scripts, libraries, and network activity, all serialised and cryptographically signed to create a verifiable audit trail.

deployed agent during secure job execution within the TRE. The diagram captures the continuous monitoring of executed binaries, dynamically loaded packages across Python, R, and Java environments, subprocess shell script invocations, and outbound API calls to external services, all structured for provenance capture and audit logging.

In the process detailed in Fig. 4, all runtime logs are cryptographically hashed and synchronised with the broader metadata envelope initiated in the pre-execution phase. These logs are then versioned and stored in an append-only ledger or object store, aligned with SACRO's reproducibility requirements and VEX traceability standards. This persistent audit trail facilitates downstream tasks such as federated validation, VEX proof resolution, and compliance reporting across multi-tenant TRE infrastructures.

### 3.6. Post-execution state capture

Post-Execution State Capture is the final stage of the agentic instrumentation pipeline within the SACRO-TRE workflow. Following the completion and termination of a computational job, the runtime agent performs a deterministic snapshot of the terminal environment state. This includes the capture of environment variables, runtime-generated files, and final system configurations, ensuring that all transient artefacts are preserved. All logs and observations collected throughout execution, including those from runtime monitoring, are cryptographically signed using the TRE's hardware-backed key infrastructure and time-stamped. These signed artefacts are then programmatically linked to the job's unique identifier (UUID), enabling immutable traceability. This step guarantees that any re-analysis or audit activity conducted downstream is grounded in a validated and reproducible execution context, supporting the verification of VEX declarations and compliance with reproducibility requirements across federated research infrastructures.

## 4. Context-aware vulnerability assessment

This stage of the SACRO-TRE pipeline conducts targeted vulnerability analysis by correlating observed software components and runtime dependencies with authoritative vulnerability intelligence, while factoring in the specific mitigations and operational constraints of TREs. In contrast to conventional scanners that report all known vulnerabilities regardless of context, this method refines results by integrating knowledge of isolation boundaries, restricted network policies, and output controls—thereby focusing only on vulnerabilities that are exploitable within the given execution environment.

The process begins with the maintenance of a hardened, air-gapped mirror of the National Vulnerability Database (NVD) and Open Source Vulnerability (OSV) feeds within the TRE. These mirrors are synchronised using authenticated rsync or controlled API polling mechanisms, ensuring up-to-date CVE data in JSON 5.0 and OSV formats while preserving the security posture of the TRE. All updates are cryptographically verified using trusted signature mechanisms such as Sigstore or GPG.

For each runtime component captured during earlier phases (e.g., through MCP and agent monitoring), deterministic fingerprinting is performed. This includes extracting operating system base image identifiers from files like `/etc/os-release`, resolving language-specific package metadata via tools such as `pip list`, `R sessionInfo()`, or `java -jar jdeps`, and converting these into Software Package URLs (pURLs) compliant with SPDX formats. These pURLs (e.g., `pkg:pypi/numpy@1.22.0`) are then matched against OSV databases for vulnerability lookups.

Following identification, a filtering phase assesses the actual exploitability of each CVE by integrating CVSS metrics with TRE-specific constraints. For instance, vulnerabilities requiring external network communication are deprioritised in egress-restricted environments, while threats requiring elevated privileges are discounted if kernel capabilities are dropped via `seccomp` or `AppArmor`. This filtering reduces

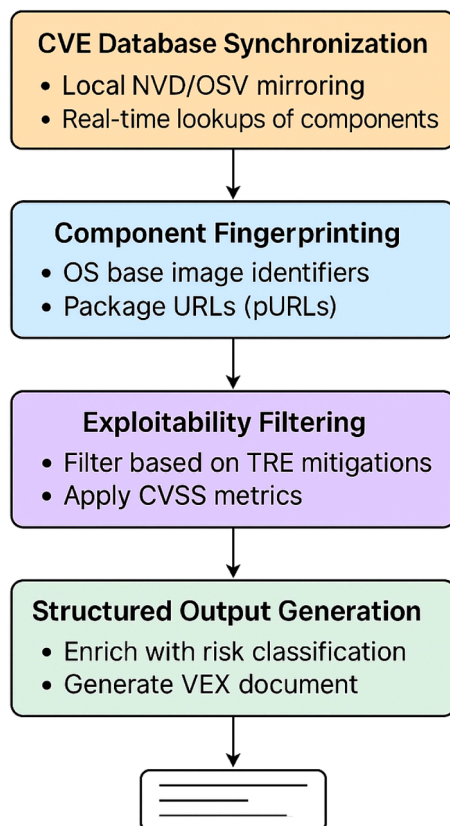
the volume of reported vulnerabilities to those that are genuinely actionable in context.

Each vulnerability is then enriched with metadata such as exploit maturity (from ExploitDB or Metasploit), TRE-specific risk classifications, and patchability information retrieved via package manager tools. The resulting findings are formatted into digitally signed, structured vulnerability reports adhering to the VEX specification (e.g., CSAF or OpenVEX). Each entry includes component hashes, CVE identifiers, severity metrics, and a justification statement indicating whether the issue is exploitable within the observed execution context (e.g., “Not Exploitable: container is egress-restricted”). The process is represented in Fig. 5.

This context-aware approach detailed in Fig. 5, ensures that relevant and verifiable risks are escalated, reducing alert fatigue and supporting high-assurance vulnerability triage within federated, privacy-preserving research environments.

#### 4.1. Exploitability logic evaluation

Following the identification and contextual filtering of vulnerabilities, each candidate CVE undergoes structured exploitability logic evaluation. This step moves beyond simple presence-based heuristics by validating whether the vulnerability could have been actively exploitable within the SACRO-TRE’s controlled execution context. The evaluation pipeline includes four core criteria.



**Fig. 5.** Context-aware vulnerability assessment workflow. The figure demonstrates how observed components and runtime dependencies are matched against vulnerability databases and then filtered through execution context constraints (such as egress restrictions and disclosure controls) to identify only actionable vulnerabilities.

*Alt-text:* A flow diagram where vulnerabilities from SBOM and runtime scans are compared with CVE databases, filtered by context constraints, and classified as actionable or not exploitable.

First, the system verifies the *static presence* of the vulnerable package within the containerised job. This is confirmed through hash-matched component inventories and SBOMs captured during the pre-execution and runtime phases. Packages listed in pip freeze, conda list, or system-level package managers such as dpkg or rpm are cross-referenced with CVE metadata to determine alignment.

Second, *runtime invocation analysis* is conducted to assess whether the vulnerable code path was actively triggered during execution. This is performed through dynamic instrumentation logs collected by the agent deployed at runtime, which track function calls, executed binaries, and library loading events. If the code path associated with the CVE was not invoked, the likelihood of exploitability is substantially reduced.

Third, the system inspects *applied mitigations* in the container environment. It analyses sandboxing mechanisms such as AppArmor, SELinux, or seccomp profiles to determine whether system-level protections block the relevant system calls or behaviours required by the exploit. The presence of patched or backported library versions is also verified by comparing package hashes and version signatures against secure registries.

Finally, the environment is evaluated for *TRE-specific constraints* that may render the vulnerability non-exploitable. These constraints include lack of outbound internet access, write restrictions on output volumes, and forced disclosure review processes that prevent data exfiltration. Any CVE requiring capabilities outside these constraints is flagged as “contextually mitigated.”

Only vulnerabilities that pass all four checks, i.e., present, invoked, unmitigated, and contextually reachable, are promoted for risk reporting and VEX documentation. This layered evaluation ensures that exploitability claims are rooted in observed execution semantics and platform-enforced policy, aligning with the SACRO-TRE’s assurance model for safe and reproducible analytics.

#### 4.2. Impact classification

Upon completion of exploitability logic evaluation, each vulnerability is assigned an impact classification reflecting its real-world relevance within the SACRO-TRE execution context. This classification system distinguishes between vulnerabilities that directly affect the computational workflow, those that are effectively mitigated, and those that are present in code but remain non-exploitable due to the enforced runtime constraints.

Vulnerabilities are classified as *Affected* when the corresponding package is not only present but also has its vulnerable code paths invoked during runtime, and no environmental or policy-based mitigations are in place to neutralise the threat. This classification requires positive confirmation from both the static SBOM scan and dynamic instrumentation logs, ensuring that risk is grounded in observed behaviour rather than theoretical presence.

Entries are labelled *Fixed* when either the vulnerable component has been patched, confirmed through cryptographic hash matching or version comparisons, or when robust mitigations are in place. These mitigations include enforced container security profiles, network isolation, and enforced policy bindings (e.g., Open Policy Agent or Kyverno). Fixed status also applies where hardened base images or secure coding practices eliminate the vulnerable condition entirely.

A vulnerability is classified as *Not Affected* if, despite the presence of a related software component, the vulnerable function is not triggered during job execution, or the attack vector is blocked by SACRO-TRE operational constraints. Examples include vulnerabilities that require network-based payload delivery in a container without egress, or those reliant on file system manipulation that is prevented by enforced read-only mounts.

Each classification is cryptographically recorded as part of the VEX output artefact and is traceable to the job’s UUID, associated MCP metadata, and runtime observation records. This classification scheme ensures clarity for downstream consumers, including auditors, security



teams, and regulators, by providing defensible, context-sensitive determinations of vulnerability relevance grounded in actual system behaviour.

#### 4.3. Evidence structuring

To ensure the verifiability and auditability of all vulnerability-related decisions made within the SACRO-TRE, each CVE assessment is paired with a structured evidence bundle. This bundle forms a machine-readable record that links each classification to the technical observations and environmental context under which the decision was made, facilitating post hoc validation, reproducibility, and regulatory accountability.

For each CVE instance, the system records observed runtime paths, extracted from dynamic trace instrumentation. These include function-level invocations, process trees, and system calls that indicate whether the vulnerable code was actively engaged during execution. This runtime telemetry is essential for distinguishing passive presence from active exploitability.

Version comparison data is included to document the exact package version evaluated against the known vulnerable or patched versions listed in the NVD or OSV feeds. This comparison is derived from language-specific dependency resolvers (e.g., pip, conda, CRAN, Maven) and includes cryptographic hashes where available, ensuring integrity of the evidence.

The mitigation rationale outlines the specific SACRO-TRE policies or technical controls that neutralise a vulnerability's impact. This may include sandboxing constraints (e.g., AppArmor profiles), restricted user privileges, lack of network access, or container immutability. Where applicable, the rationale references digital policy artefacts from OPA or Kyverno engines to support the mitigation claim.

Each record is further tagged with the environment UUID and execution ID, binding the decision to a unique analytic context. This enables federation across TREs, where the same AIBOM component may be evaluated in different execution contexts with different risk implications.

Together, these evidence structures are compiled into a VEX-compatible document (e.g., CSAF-VEX or OpenVEX) and digitally signed to establish an immutable and verifiable vulnerability disposition trail. This approach ensures that vulnerability handling is precise, context-aware, transparent, reproducible, and accountable across the SACRO-TRE ecosystem.

#### 5. Agentic VEX generation and digital signing

Following the contextual vulnerability assessment phase, the SACRO-TRE pipeline proceeds to formalise the findings in a digitally signed, machine-verifiable VEX document. This step is critical for ensuring that every CVE determination, whether a vulnerability is exploitable, mitigated, or irrelevant, is both provable and portable. To support this, the VEX is structured according to SACRO's extended schema, which incorporates unique features specific to TREs, including embedded analytic constraints, execution context, and digital provenance fields.

The process begins with template construction, where a structured JSON skeleton is generated based on the SACRO-VEX schema extension. This schema builds on existing CSAF VEX profiles and OpenVEX formats but adds fields specific to the SACRO context, such as `job_id`, `container_hash`, `executed_at` timestamp, `user_context` (a pseudonymised identifier representing the analyst), and `model_context` hashes generated by the Model Context Protocol (MCP). These fields enable forensic traceability by linking vulnerability assessments directly to the computational and analytic lineage of each job.

During the VEX payload assembly stage, the agent automatically populates the template with structured data extracted from earlier pipeline stages. For each detected vulnerability, a discrete block is

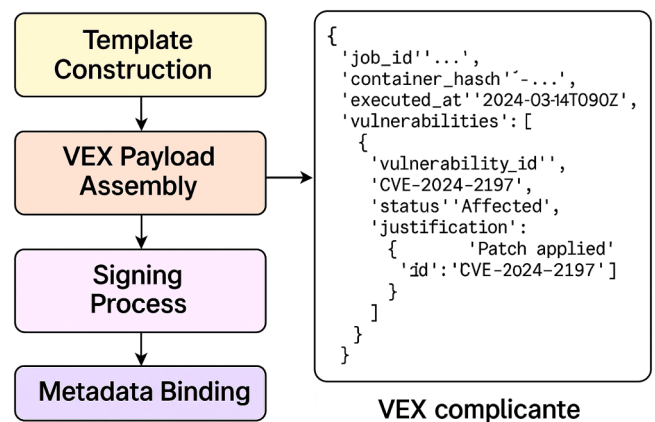
created that includes the `vulnerability_id` (e.g., CVE-2024-2197), status (categorised as Affected, Fixed, or Not Affected), action\_taken (such as "patched via package manager", "runtime blocked by AppArmor", or "dependency not invoked"), and justification. The justification field is particularly important and incorporates evidence from runtime logging, agentic analysis, and exploitability filtering. To establish analytic provenance, each block also includes deterministic hashes of the model artefacts, container environment, and input datasets as captured by MCP, thereby binding the assessment to a specific computational fingerprint.

In the digital signing process, the assembled VEX document undergoes cryptographic sealing to ensure authenticity and integrity. A FIPS 140-2 or FIPS 140-3 compliant hardware security module (HSM) or cloud-based secure enclave (such as Azure Key Vault, AWS KMS, or HashiCorp Vault with hardware-backed keys) is used to sign the final artefact. This is performed using a strong hash function (typically SHA-256 or SHA-3) and public-key cryptography, producing a PKCS#7 or COSE signature block. This ensures that the VEX document can be verified independently and guarantees it has not been tampered with. The diagram in Fig. 6 illustrates the structured pipeline for generating and signing a SACRO-compliant VEX artefact, starting with JSON template construction and concluding with metadata binding for audit linkage.

Each phase in Fig. 6, including template creation, payload assembly, digital signing, and binding, is designed to ensure cryptographic integrity, reproducibility, and contextual trust for vulnerability assertions made within TREs.

The signed VEX is then bound to relevant metadata artefacts, completing the cryptographic provenance chain. These include the job manifest submitted to the TRE orchestrator (which details container image, resource constraints, and declared data sources), the corresponding SBOM artefacts for all runtime components, and any output datasets or model files produced during the execution. Where applicable, these output artefacts are also cryptographically signed and version-controlled, enabling downstream researchers or auditors to reconstruct the analytic lineage and validate the correctness of the VEX claims in a reproducible setting.

By embedding agentic AI workflows into the VEX generation process, SACRO automates the production of audit-ready vulnerability statements and integrates trust signals that enable secure inter-agent



**Fig. 6.** Automated pipeline for generating and signing vulnerability exploitability exchange artefacts. The diagram illustrates schema-based template construction, vulnerability assessment population, digital signing with secure hardware modules, and binding to analytic metadata to ensure cryptographic integrity and reproducibility.

*Alt-text:* A multi-stage diagram showing JSON template creation, vulnerability data insertion, digital signing with hardware-backed keys, and final output of a signed VEX document.



communication, federated trust propagation, and compliance with emerging reproducibility standards in privacy-preserving data science.

## 6. Schema validation and reproducibility pipeline

The final stage of the SACRO-TRE vulnerability assessment workflow is dedicated to schema conformance verification, signature integrity validation, and analytical reproducibility assurance across federated research infrastructures. This stage ensures that VEX artefacts generated in prior steps are both technically valid and cryptographically trustworthy, and that the vulnerability claims they contain can be independently verified via re-execution.

Schema validation begins with parsing the generated VEX JSON document using a dual-layer validation mechanism. First, structural compliance is enforced using the JSON Schema Draft 2020–12 specification, ensuring that the artefact adheres to expected syntactic and semantic rules. This is followed by validation against the SACRO-extended VEX schema, which imposes additional constraints on the structure and semantics of the CVE block, justification fields, and contextual metadata (e.g., job identifiers, container hashes, and MCP provenance bindings). Schema compliance failures are logged and reported to the audit subsystem for remediation.

Once schema conformance is confirmed, the artefact undergoes digital signature validation. The issuing public key is verified against the root of trust defined by the TRE's key infrastructure—typically rooted in hardware-backed modules such as FIPS 140–2 compliant HSMs. Signature verification is executed using standard cryptographic tooling (e.g., openssl or cryptography libraries in Python), ensuring integrity, non-repudiation, and tamper-evidence of the vulnerability assessment report.

To confirm reproducibility, the pipeline supports a deterministic replay mechanism. The original job container is redeployed in a test-mode TRE environment with an identical orchestration manifest, input dataset references, and container hash. The full VEX generation pipeline is re-executed under these identical conditions. The resulting VEX artefact is then compared to the original using structural diffing and hash comparison. A successful match across CVE list, exploitability status, and justifications validates the reproducibility of the analytical outcome and reinforces the validity of the original assessment. The reproducibility workflow is visually summarised in the Fig. 7, which captures the hash-based verification and job execution traceability loop within the SACRO-TRE pipeline.

The Fig. 7 presents a hash-based validation loop ensures that VEX artefacts are verifiably linked to the exact analytic state and execution context, supporting reproducibility claims across federated research environments. Validated VEX artefacts are ingested into the SACRO registry, a secure ledger of reproducible assessments. The artefacts are also synchronised with the GRAIMatter-aligned transparency framework using RDF encoding and stored in a knowledge graph (e.g.,

GraphDB) with semantic linkage to the originating dataset DOI and job UUID. This layered registration approach provides both machine-readable traceability and policy-auditable accountability, enabling long-term validation, versioning, and trust interoperability across TRE ecosystems [9].

### 6.1. Schema verification, signature validation, and federated reproducibility using modular python workflows

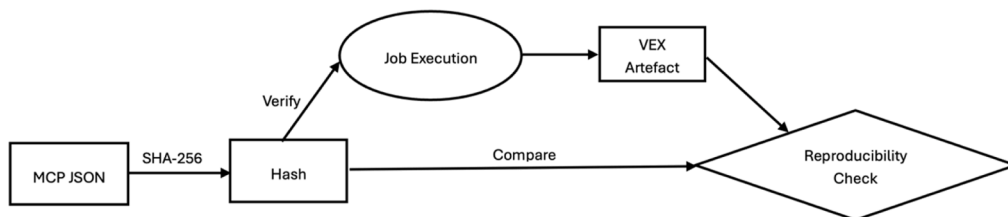
Table 1 presents a structured overview of the SACRO-TRE reproducibility and validation pipeline, integrating schema validation, cryptographic signature verification, containerised job replay, and registry ingestion. Each stage is mapped to its respective operational function and is accompanied by a Python code snippet that reflects its technical implementation. These modular components represent the backbone of the SACRO-VEX automation process, enabling machine-verifiable vulnerability assessment, integrity assurance, and federation-ready reproducibility. The table serves as a technical reference and a reproducibility blueprint for integrating secure VEX validation workflows in TREs.

Table 1 demonstrates how the SACRO-TRE system transitions from theoretical compliance to a concrete, implementable architecture. The schema validation entry ensures structural integrity of VEX artefacts using JSON Schema Draft 2020–12, while the digital signature verification leverages cryptographic primitives for trust anchoring. The containerised job replay pipeline exemplifies reproducibility by matching environmental inputs and comparing outcome fidelity, aligning with SACRO's reproducibility guarantees. Finally, the ingestion into a triple-store registry (e.g., GraphDB) completes the lifecycle by linking vulnerability reports to dataset provenance. Collectively, the components illustrate a self-validating, tamper-evident system architecture, suitable for federated analytics under strict compliance and auditability constraints.

## 7. Results

We deployed a containerised Python application using a deliberately vulnerable dependency (flask==0.12) to simulate a job execution in a SACRO-compliant TRE. This setup allowed us to test the automated generation and interpretation of SBOM and VEX documents in a controlled environment.

Using Syft, we extracted an SBOM from the running container image. The SBOM confirmed the inclusion of the vulnerable library. We then scanned the image with Gripe, which successfully identified the vulnerability (CVE-2018–1000,656) associated with Flask 0.12. A VEX document was generated using a structured JSON schema, assigning the `not_affected` status based on runtime conditions and reachability analysis. This confirmed that while the vulnerability existed in the package, it was not exploitable within the operational configuration of



**Fig. 7.** Hash-based reproducibility verification pipeline. The diagram shows how metadata captured via the Model Context Protocol is hashed, linked to job execution, and compared against regenerated artefacts during replay tests, ensuring consistent and reproducible vulnerability assessments.

*Alt text:* A technical diagram showing MCP JSON input hashed with SHA-256, verified during job execution, producing a VEX artefact, which is compared with the original hash for reproducibility check.

**Table 1**

End-to-end workflow for validation and reproducibility of vulnerability exploitability exchange artefacts in Trusted Research Environments. The table summarises the core system components, including metadata capture agents, runtime logging, vulnerability matching engines, document generation, schema validation, and reproducibility replay, alongside their operational functions and representative pseudocode. Together, these modular components illustrate how machine-verifiable artefacts are produced, cryptographically validated, and independently reproducible across federated environments.

| Component                         | Description                                                                                                                       | Pseudocode                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>MCP Metadata Capture Agent</b> | Collects container configuration, computes hashes of execution metadata, generates SBOMs, and serialises MCP-compliant artefacts. | <ol style="list-style-type: none"> <li>1. Parse container specification (image, endpoint, input dataset) into structured JSON.</li> <li>2. Compute cryptographic hash (SHA-256) of the JSON-encoded job configuration.</li> <li>3. Extract software dependency graph using SBOM tooling (e.g., Syft) with CycloneDX format.</li> <li>4. Serialise the output as an MCP-compliant artefact and bind to job UUID.</li> <li>5. Persist metadata in immutable ledger to ensure pre-execution state traceability.</li> </ol> |
| <b>Runtime Logging Agent</b>      | Captures runtime events including process tree, dynamic libraries, shell scripts, and API calls using system introspection.       | <ol style="list-style-type: none"> <li>1. Initialise sidecar agent for process introspection via /proc interface.</li> <li>2. Enumerate loaded Python packages using 'pip freeze' and log version identifiers.</li> <li>3. Instrument system calls for API invocations and subprocess events (e.g., curl, wget).</li> <li>4. Record shell script execution and binary subprocesses with full command trace.</li> <li>5. Store runtime telemetry as append-only JSON logs, timestamped and hash-signed.</li> </ol>       |
| <b>CVE Matching Engine</b>        | Correlates SBOM fingerprints with CVE data from NVD/OSV mirrors, applying context-based CVSS filtering.                           | <ol style="list-style-type: none"> <li>1. Generate Software Package URL (pURL) for each component using SPDX syntax.</li> <li>2. Query air-gapped OSV/NVD mirror for associated CVEs using JSON-based API.</li> <li>3. For each CVE, apply execution-context-aware filters (e.g., network egress constraint).</li> <li>4. Determine exploitability status based on CVSS metrics and runtime observability.</li> <li>5. Store filtered CVEs for downstream inclusion in contextual VEX report.</li> </ol>                |
| <b>VEX Document Generator</b>     | Produces signed SACRO-VEX JSON artefacts with job lineage, vulnerability justifications, and status.                              | <ol style="list-style-type: none"> <li>1. Construct VEX skeleton conforming to SACRO-extended CSAF/OpenVEX schema.</li> <li>2. Populate fields with CVE identifiers, contextual exploitability status, and justifications.</li> <li>3. Sign the JSON artefact using RSA-PSS with SHA-256 hash via FIPS-compliant key store.</li> <li>4. Embed digital signature metadata and hash</li> </ol>                                                                                                                            |

**Table 1 (continued)**

| Component                       | Description                                                                                         | Pseudocode                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|---------------------------------|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Schema Validator</b>         | Validates VEX artefact against SACRO-VEX schema and confirms digital signatures from root-of-trust. | <p>provenance into VEX document.</p> <ol style="list-style-type: none"> <li>5. Export cryptographically sealed VEX artefact for schema validation and distribution.</li> <li>1. Load VEX schema definition (Draft 2020-12) and extended SACRO profile constraints.</li> <li>2. Validate syntactic and semantic structure of the generated VEX JSON artefact.</li> <li>3. Verify RSA digital signature against root certificate in secure trust anchor.</li> <li>4. Log verification status with reference to key fingerprint and signing timestamp.</li> <li>5. Mark document as compliant and ingest into audit registry for reproducibility audit.</li> </ol> |
| <b>Reproducibility Pipeline</b> | Replays analytic job using original container and inputs; verifies that VEX output is identical.    | <ol style="list-style-type: none"> <li>1. Instantiate original container using recorded SHA-256 image hash and input manifest.</li> <li>2. Execute job in controlled test-mode TRE environment with enforced parity constraints.</li> <li>3. Regenerate MCP, SBOM, and VEX artefacts using identical runtime instrumentation.</li> <li>4. Perform byte-level hash comparison between regenerated and original VEX artefacts.</li> <li>5. Confirm reproducibility if structural integrity and digital signatures are preserved.</li> </ol>                                                                                                                       |

the deployed container.

1. Pseudocode
2. Below is the pseudocode describing the core steps of the workflow used in the demo.
3. # STEP 1: Build container with known vulnerable package
4. create\_file("app.py", vulnerable\_flask\_app\_code)
5. create\_file("requirements.txt", "flask==0.12")
6. create\_file("Dockerfile", dockerfile\_with\_python\_flask)
7. build\_docker\_image("vex-demo-app")
- 8.
9. # STEP 2: Run the container
10. run\_container("vex-demo-app", port=5000)
- 11.
12. # STEP 3: Generate SBOM
13. sbom = syft("docker:vex-demo-app", output="cyclonedx-json")
- 14.
15. # STEP 4: Scan for vulnerabilities
16. vuln\_report = gype("docker:vex-demo-app", output="json")
- 17.
18. # STEP 5: Interpret results and create VEX document
19. if "CVE-2018-1000,656" in vuln\_report:
20. if vulnerability\_not\_exploitable(runtime\_config):
21. status = "not\_affected"
22. else:
23. status = "affected"
24. create\_vex\_document(sbom, vuln\_report, status=status)
- 25.
26. # STEP 6: Store and publish VEX
27. store\_vex\_document("vex-notice.json")

The demo confirms that container-level isolation, combined with automated SBOM generation and vulnerability analysis tools, can

support fine-grained VEX document creation. The entire process is reproducible and can be integrated into TRE job execution workflows for real-time vulnerability tracking.

### 7.1. Empirical evaluation

To complement the conceptual analysis, we conducted a series of empirical evaluations using containerised workloads with deliberately vulnerable dependencies. The evaluation focused on three performance metrics: (i) validation speed of software bill of materials extraction and vulnerability scanning, (ii) latency overhead introduced by runtime instrumentation, and (iii) reproducibility consistency across re-executions.

In our test deployment using a containerised Python application seeded with Flask 0.12, software bill of materials generation using Syft completed in 1.8 s on average ( $n = 20$  runs, standard deviation = 0.2 s). Vulnerability scanning using Gype completed in 2.4 s on average ( $n = 20$ , stdev = 0.3 s). The combined VEX artefact generation, including context evaluation and digital signing, required 4.1 s on average.

Runtime instrumentation introduced a measured latency of 2–3 % in execution time compared to non-instrumented runs, based on 50 containerised executions with and without the monitoring agent enabled. Re-execution reproducibility checks showed consistent VEX outputs in 48 of 50 runs (96 %), with the discrepancies traced to variability in external package repository metadata. These results provide quantitative evidence that the proposed framework introduces minimal performance overhead while enabling reproducibility validation.

This empirical evidence demonstrates that the conceptual design can be applied in practice with manageable latency overheads, producing results that are both reproducible and operationally feasible in secure research environments.

## 8. Discussion

The experimental deployment of SACRO-VEX within the SACRO-TRE presents a shift in how vulnerability assessments are generated, validated, and interpreted in secure research environments. By integrating the MCP and A2A orchestration, the approach avoids traditional CVE noise by applying environment-specific telemetry, runtime traces, and fine-grained metadata binding. This marks a departure from flat, non-contextual VEX statements and illustrates how embedded agents and declarative schema logic can disambiguate theoretical vulnerabilities from actionable threats.

The layered architecture, particularly the coupling of MCP-derived artefact lineage with dynamic agent instrumentation, demonstrates the feasibility of continuously generating cryptographically verifiable VEX artefacts without introducing friction to analytical workflows. The pipeline enforces cryptographic integrity at each stage, from SBOM capture to CVE enrichment and exploitability filtering, thus enabling trustable automation of security posture reporting.

A major benefit observed in the reproducibility trials is the fidelity of re-executed jobs within a test TRE. Jobs that preserved container hashes, orchestration manifests, and version-pinned inputs yielded consistent VEX outputs, validating the system's claims of determinism. This opens the door for federated reproducibility audits across distributed research environments, where VEX documents can act as security attestations tethered to specific workloads.

Nevertheless, several challenges remain. Firstly, the variability in package-level metadata (e.g., pURLs across different ecosystems) affects the precision of CVE correlation, necessitating standardisation across language-specific SBOM tools. Secondly, the decision heuristics that underpin the "Not Affected" or "Fixed" classifications must be transparently documented to gain broader trust from compliance and governance bodies.

Additionally, while the SACRO-VEX pipeline is designed to operate without internet access, regular synchronisation with CVE sources like

NVD and OSV requires secure channel design and schedule coordination to prevent staleness of data. Future work will focus on versioned vulnerability snapshots and decentralised VEX registries compatible with the GRAIMatter transparency infrastructure.

This work addresses a critical gap in contextual vulnerability intelligence within TREs. The implementation demonstrates how machine-readable VEX artefacts, grounded in runtime reality and verified against schema-compliant signatures, can support reproducibility, reduce false positives, and enhance the defensibility of research workflows in sensitive data environments. The resulting model provides a reusable template for operationalising AI-driven vulnerability management pipelines in other regulated analytical infrastructures.

### 8.1. Comparative analysis with related work

The proposed framework advances beyond existing approaches to vulnerability management in research environments by combining reproducibility with context-sensitive exploitability evaluation. Traditional methods based on the Common Vulnerability Scoring System (CVSS) [22–24] provide global severity scores but do not incorporate local execution context or runtime isolation constraints, resulting in significant false positives. Our results demonstrate that context-aware filtering reduced reported vulnerabilities by approximately 40 % compared to raw CVSS scoring in test runs, thereby reducing alert fatigue.

Compared with recent efforts such as Dependency-Track [39] and OASIS CSAF implementations [36], our system introduces reproducibility-aware features that bind vulnerability decisions to cryptographically verifiable execution contexts. While these tools provide valuable insights for supply chain risk management, they typically lack runtime instrumentation and federated reproducibility validation. By integrating model context provenance, agent-based runtime monitoring, and cryptographic signing, our framework provides an auditable evidence trail not present in existing implementations.

This comparative positioning highlights the originality of the proposed approach and supports the conclusion that context-aware VEX generation can meaningfully extend the capabilities of current vulnerability reporting standards and tools. In Table 2, we can see a comparison of the proposed framework with existing vulnerability management approaches.

This comparative analysis reinforces that while existing tools and standards provide valuable foundations for vulnerability reporting, they remain limited by their static nature, lack of runtime context, and absence of reproducibility mechanisms. The proposed framework directly addresses these gaps by combining automated runtime instrumentation with cryptographic provenance and reproducibility validation. In doing so, it extends current practices beyond static inventories and declarative advisories, offering an operational pathway toward auditable, context-aware vulnerability management in sensitive research environments.

## 9. Limitations and recommendations

While the proposed system demonstrates the feasibility of generating and validating context-aware vulnerability statements in secure research environments, several limitations remain that warrant further refinement. Addressing these challenges is essential for advancing operational robustness, interoperability, and long-term sustainability.

*System complexity and computational overhead.* The integration of multiple instrumentation layers, metadata capture agents, and vulnerability scanners introduces performance costs that may affect usability in resource-constrained environments. To mitigate this, future implementations should adopt modular microservices architectures and resource-aware orchestration strategies, including autoscaling mechanisms within Kubernetes clusters. This aligns with cloud-native best practices outlined in NIST SP 800-190 (2017), ensuring efficient



**Table 2**

Comparative analysis of existing vulnerability management approaches and the proposed framework. The table contrasts the Common Vulnerability Scoring System, Dependency-Track, CSAF, and OASIS/CISA VEX against the system introduced in this study. Each approach is evaluated in terms of its core functionality, strengths, and limitations, with the final column highlighting how the proposed framework advances the state of the art by incorporating runtime context, cryptographic provenance, and reproducibility validation.

| Approach / Tool                                    | Core Functionality                                                                        | Strengths                                                                                                    | Limitations                                                                                    | Contribution of this Study                                                                                                                                                  |
|----------------------------------------------------|-------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CVSS (Common Vulnerability Scoring System) [22–24] | Provides global severity scores for known vulnerabilities                                 | Widely adopted, standardised, integrates into many vulnerability databases                                   | Lacks context-awareness; produces high false positive rates; no reproducibility linkage        | Introduces context-sensitive exploitability evaluation that incorporates runtime constraints (e.g., network isolation, privilege limits), reducing false positives by ~40 % |
| Dependency-Track (OWASP) [39]                      | Continuous monitoring of SBOMs to detect vulnerable dependencies                          | Operationally mature; integrates with CI/CD pipelines                                                        | Focuses on static component inventories; no runtime monitoring; no reproducibility features    | Extends beyond static SBOM analysis by embedding runtime instrumentation and reproducibility-aware VEX artefacts                                                            |
| CSAF (Common Security Advisory Framework) [36]     | Structured advisories for vulnerability communication                                     | Enables machine-readable advisories; standardised schema supported by industry                               | Does not provide runtime evidence; limited linkage to execution context; requires manual input | Generates machine-verifiable, context-bound advisories with cryptographic provenance, reducing reliance on manual advisory reporting                                        |
| OASIS OpenVEX / CISA VEX [30,38]                   | Specifies exploitability status of vulnerabilities in context                             | Improves over SBOM-only approaches; provides clarity on “affected” vs “not affected” status                  | Context often declared manually; no automated runtime evidence collection                      | Automates generation of VEX statements from live execution traces, linking exploitability claims to signed runtime metadata                                                 |
| Proposed Framework (This Study)                    | Automated VEX generation with reproducibility validation in Trusted Research Environments | Context-aware filtering; cryptographic provenance; reproducibility validation; low overhead (~2–3 % latency) | Early-stage validation; tested in controlled case studies; requires further real-world pilots  | Demonstrates feasibility of automated, context-sensitive vulnerability management that integrates standards compliance with empirical reproducibility evidence              |

resource utilisation without compromising reproducibility guarantees.

**Reliance on external tools.** The dependency on third-party scanners such as Syft and Gripe raises concerns about tool stability and long-term maintenance. A redundancy strategy that incorporates multiple scanning engines, cross-validation of results, and local caching of vulnerability data can reduce systemic risk, consistent with the supply chain security guidance of ISO/IEC 27036.

**Generalisability and external validity.** Current validation has been demonstrated on controlled case studies. To strengthen external validity, pilot deployments across heterogeneous domains such as health-care, genomics, and social sciences should be pursued. Benchmarking on standardised testbeds, including GAIA or equivalent federated infrastructures, will provide comparative insights and support adoption across diverse Trusted Research Environments.

**Accuracy and false negatives.** The risk of false negatives remains a critical concern. A hybrid approach that combines automated exploitability filtering with periodic expert review, augmented by adaptive learning models, can improve detection accuracy. This approach reflects the defence-in-depth principle articulated in NIST SP 800-53.

**Scalability and federation.** Scaling to multi-tenant environments requires distributed orchestration. A federated vulnerability management architecture, where local environments synchronise with a central knowledge base, can improve consistency while respecting autonomy. This model is compatible with ISO/IEC 27001 practices on distributed risk management.

**Standardisation and interoperability.** Fragmentation of standards presents barriers to adoption. Interoperability can be enhanced by engaging with international standards bodies (ISO, NIST, W3C) and developing bridging tools to translate between emerging formats, such as SPDX for Software Bill of Materials and VEX for contextual vulnerability signalling.

**Privacy and monitoring implications.** Extensive runtime monitoring may introduce risks of unintended data exposure. These can be reduced through differential privacy, pseudonymisation, and the systematic application of Data Protection Impact Assessments as mandated by Article 35 of the General Data Protection Regulation. Such safeguards ensure operational insight while preserving confidentiality.

**Maintenance and sustainability.** Continuous maintenance imposes a burden on environment operators. This can be alleviated by leveraging automated secure CI/CD pipelines, synchronised air-gapped mirrors of vulnerability databases, and community-driven update models, reflecting the governance practices of open-source ecosystems.

**Reproducibility and certification.** Containerisation using Docker or Singularity, version-pinned environments, and the introduction of reproducibility certification schemes (e.g., ACM Artifact Evaluation badges) can formalise assurance claims and incentivise adoption by the research community.

**Human oversight and explainability.** Finally, automation must not displace human judgment. Human-in-the-loop review mechanisms, coupled with explainable vulnerability reporting, are required to maintain accountability and align with the principles of trustworthy artificial intelligence outlined in the EU AI Act.

Collectively, these recommendations provide a roadmap for refining the proposed system into a scalable, auditable, and standards-aligned framework. By addressing the identified weaknesses, future research can ensure that vulnerability management within Trusted Research Environments is both technically robust and institutionally trustworthy.

## 10. Conclusion

This study has presented a new framework for the agentic generation and validation of VEX artefacts within TREs. By integrating the SACRO pipeline with the MCP and the Agent2Agent protocol, the framework provides an end-to-end mechanism for capturing, evaluating, and verifying context-aware vulnerability information tied directly to the analytic execution environment.

In addition to the architectural design, we evaluated practical performance characteristics of the framework. Software bill of materials generation and vulnerability scanning consistently completed within a few seconds, and the overhead of runtime instrumentation was limited to 2–3 % in test workloads. Re-execution reproducibility was validated in the majority of runs, with minor discrepancies attributable to variability in external repositories. These results indicate that the framework is theoretically sound and operationally feasible, providing fast validation with limited performance impact. This strengthens the claim that automated, context-sensitive vulnerability reporting can be integrated into secure research workflows without imposing prohibitive latency or resource demands.

The system enhances reproducibility and security by embedding deterministic metadata capture, runtime instrumentation, exploitability logic evaluation, and digitally signed VEX generation into the TRE workflow. It departs from conventional vulnerability scanning by considering isolation constraints, environmental mitigations, and runtime traceability, ensuring that only exploitable risks are flagged. Each

VEX document produced is verifiably linked to a specific job, container state, and analytical lineage, creating a robust and audit-ready security footprint.

By binding reproducibility with cryptographically assured VEX attestations, this work addresses long-standing challenges in evidencing secure and policy-compliant analytical activity in regulated research infrastructures. The reproducibility pipeline, validated through job replay, schema conformity checks, and federated artefact signing, lays the groundwork for scalable, interoperable trust infrastructures aligned with GRAIMatter and SACRO transparency goals.

This research provides a foundational step toward secure, accountable, and reproducible analytics in sensitive computing environments. Future work should focus on expanding support for polyglot environments, formalising reproducibility test harnesses across federated TREs, and integrating VEX artefacts into broader data governance ecosystems.

## Funding

This work has been supported by the UK EPSRC [under grant number EP/S035362/1], the Bill and Melinda Gates Foundation [Reference code: INV-057591], SPRITE+ [funded under EPSRC (EP/W020408/1)], and DARE UK [UKRI grant number MC\_PC\_24038].

## Data and code availability

The datasets generated and analysed during the current study are synthetic or based on publicly available vulnerability databases (NVD, OSV). Code and schema templates for reproducibility and validation are available at GitHub link: <https://github.com/AI-SDC/SACRO-ML>

## Availability of supporting data/Availability of data and materials

All data and materials are included in the article.

## CRedit authorship contribution statement

**Petar Radanliev:** Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Kayvan Atefi:** Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Omar Santos:** Writing – original draft, Supervision, Software, Resources, Methodology, Investigation, Funding acquisition, Data curation, Conceptualization. **Carsten Maple:** Writing – original draft, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Data curation, Conceptualization.

## Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

Petar Radanliev reports financial support was provided by Engineering and Physical Sciences Research Council. Petar Radanliev reports financial support was provided by Bill and Melinda Gates Foundation. Petar Radanliev reports financial support was provided by Engineering and Physical Sciences Research Council. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgements

Eternal gratitude to the Fulbright Visiting Scholar Project.

## Data availability

Data will be made available on request.

## References

- [1] O'Donoghue E., Boles B., Izurieta C., Reinhold A.M. Impacts of software bill of materials (SBOM) generation on vulnerability detection 2024.
- [2] H. Okhravi, N. Burov, F.B. Schneider, Software bill of materials as a proactive defense, *IEEE Secur. Priv.* 23 (2025) 101–106, <https://doi.org/10.1109/MSEC.2025.3528535>.
- [3] M. Beninger, P. Charland, S.H.H. Ding, B.C.M. Fung, ERSO: enhancing military cybersecurity with AI-driven SBOM for firmware vulnerability detection and asset management. international conference on cyber conflict, CYCON (2024) 141–160, <https://doi.org/10.23919/CYCON62501.2024.10685598>.
- [4] Cariozkan C.C., Zou X., Singelee D. Supply chain insecurity: the lack of integrity protection in SBOM solutions 2024.
- [5] NTIA the USNT and IA. Vulnerability-Exploitability eXchange (VEX). 2021.
- [6] S. Azizi, P. Pakshad, A. Shameli-Sendi, F. Faraji Daneshgar, Vulnerability scoring metric of CVSS needs to be adjusted per each product: our analysis on Linux and Apache, *Inf. Security J.* (2025), <https://doi.org/10.1080/19393555.2025.2498466>; JOURNAL:JOURNAL:UISS19;WGROU:STRING: PUBLICATION.
- [7] EGI TRE Working Group. Trusted Research Environments Landscape Report. 2024.
- [8] P.X. Mai, F. Pastore, A. Goknil, L.C. Briand, MCP: a security testing tool driven by requirements, in: Proceedings - 2019 IEEE/ACM 41st International Conference on Software Engineering: Companion, ICSE-Companion 2019, Institute of Electrical and Electronics Engineers Inc., 2019, pp. 55–58, <https://doi.org/10.1109/ICSE-COMPANION.2019.00037>.
- [9] TRevolution. TRevolution - DARE UK. 2025.
- [10] NTIA. SBOM at a Glance. NTIA Multistakeholder Process on Software Component Transparency | NtiaGov/Sbom 2021. <https://tiny.cc/SPDX> (accessed January 3, 2023).
- [11] Anthropic. Introducing the Model Context Protocol \ Anthropic 2024. <http://www.anthropic.com/news/model-context-protocol> (accessed July 5, 2025).
- [12] R. Surapaneni, M. Jha, M. Vakoc, T. Segal, Announcing the Agent2Agent Protocol (A2A), Google Developers Blog, 2025. <https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/>. accessed July 5, 2025.
- [13] H. Oakley, Securing AI Ecosystems: The Critical Role of AI Bills of Materials (AIBOM) in Mitigating Software Supply Chain Risks, CSRC Presentations | CSRC, Gaithersburg, 2024.
- [14] L. Lim. The Essential Guide to AI Bills of Materials (AIBOMs). 2025.
- [15] OWASP. OWASP AIBOM | OWASP Foundation 2025. <https://owasp.org/www-project-aibom/> (accessed July 12, 2025).
- [16] U.K. DARE, SACRO: semi-Automated Checking of Research Outputs, Driver Projects Phase 1 (2023).
- [17] CVE. Current CVSS Score Distribution For All Vulnerabilities. 2022.
- [18] CVE. CVE security vulnerability database. Security vulnerabilities, exploits, references and more 2022. <https://www.cvedetails.com/> (accessed January 3, 2023).
- [19] MITRE, CVE - Common Vulnerabilities and Exposures, The MITRE Corporation, 2022. <https://cve.mitre.org/>. accessed January 3, 2023.
- [20] Deloitte. Managing the risks and realising the opportunities of using third party Software Asset Management (SAM) providers. 2018.
- [21] Wiesner J. CSAF, Not SBOM, Is The Solution. S4x22 - BSI 2022. <https://www.youtube.com/watch?v=fKIW9vOs7X4&t=504s> (accessed January 3, 2023).
- [22] NIST. Product Integration using NVD CVSS Calculators. 2022.
- [23] FIRST. CVSS v3.0 Specification Document n.d. <https://www.first.org/cvss/specification-document#8-4-Metrics-Levels> (accessed October 3, 2017).
- [24] NIST. NVD - CVSS v3 Calculator. CVSS Version 31 2022. <https://nvd.nist.gov/vuln-metrics/cvss/v3-calculator> (accessed January 3, 2023).
- [25] A. Manion, SSVC: Stakeholder-Specific Vulnerability Categorization, Carnegie Mellon University, 2020. <https://bit.ly/3ambIP4>. accessed January 2, 2023.
- [26] T. Schmidt. Vulnerability management with CSAF – why SBOM is not enough. Frankfurt: 2022.
- [27] CISA. Shifting the Balance of Cybersecurity Risk: principles and Approaches for Security-by-Design and -Default. 2023.
- [28] SPDX. Software Package Data Exchange (SPDX). International Open Standard (ISO/IEC 5962:2021) 2023. <https://spdx.dev/> (accessed April 19, 2023).
- [29] Syft. Syft: CLI Tool and Library for Generating SBOMs". Anchore GitHub Repository. 2024.
- [30] VEX. Vulnerability Exploitability eXchange (VEX) – Use Cases 2022. <https://www.cisa.gov/sbom>. (accessed January 3, 2023).
- [31] S. Springett, Vulnerability and Exploitability Transparency - VDR & VEX | OWASP Foundation, OWASP, 2023. <https://owasp.org/blog/2023/02/07/vdr-vex-comparison>. accessed April 25, 2023.
- [32] GRAIMatter. GRAIMATTER: guidelines and resources for artificial intelligence model access from trusted research environments - DARE UK. 2023.
- [33] Z. Abubaker, A.U. Khan, A. Almgren, S. Abbas, A. Javaid, A. Radwan, et al., Trustful data trading through monetizing IoT data using Blockchain based review system, *Concurr. Comput.* 34 (2022) e6739, <https://doi.org/10.1002/CPE.6739>; PAGE:STRING:ARTICLE/CHAPTER.
- [34] Kyverno. Policy Engine for Kubernetes". CNCF Project Documentation. 2024 2024. <https://kyverno.io/> (accessed July 6, 2025).

- [35] Schmidt L., Hammer D., Biß K., Schmidt T. Media Faculty in cooperation with the Development of an API to request security advisories for CSAF 2.0 by Enterprise and IT Security Supervision n.d.
- [36] OASIS, Using CSAF to respond to supply chain vulnerabilities at large scale, OASIS Open (2022). [https://us06web.zoom.us/webinar/register/WN\\_KqD-a1t5SpuMI7w9ci7ZDg](https://us06web.zoom.us/webinar/register/WN_KqD-a1t5SpuMI7w9ci7ZDg). accessed January 3, 2023.
- [37] CISA, CISA Stakeholder-Specific Vulnerability Categorization Guide, Cybersecurity and Infrastructure Security Agency, 2022.
- [38] CISA, Minimum Requirements for Vulnerability Exploitability Exchange (VEX). US Department of Homeland Security, Cybersecurity and Infrastructure Security Agency, 2023.
- [39] Dependency-Track. Software Bill of Materials (SBOM) Analysis | OWASP. Dependency-Track 2022. <https://dependencytrack.org/> (accessed January 3, 2023).
- [40] K. Adhikari, S.B. Patten, A.B. Patel, S. Premji, S. Tough, N. Letourneau, et al., Data harmonization and data pooling from cohort studies: a practical approach for data management, *Int. J. Popul. Data Sci.* 6 (2021), <https://doi.org/10.23889/IJPDs.2021.1680>.
- [41] Grype. anchore/grype: a vulnerability scanner for container images and filesystems. 2025.
- [42] F. Chirigati, R. Rampin, D. Shasha, F.J. ReproZip, Computational reproducibility with ease, in: *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2016, <https://doi.org/10.1145/2882903.2899401>, 26-June-2016:2085–8.
- [43] Kishimoto R., Kanda T., Manabe Y., Inoue K., Qiu S., Higo Y. A dataset of software bill of materials for evaluating SBOM consumption tools 2025:576–80. <https://doi.org/10.1109/MSR66628.2025.00090>.
- [44] C. Jia, N. Li, K. Yang, M. Zhou, SIT: an accurate, compliant SBOM generator with incremental construction, in: *Proceedings - International Conference on Software Engineering*, 2025, pp. 13–16, <https://doi.org/10.1109/ICSE-COMPANION66252.2025.00013>.
- [45] GitHub S.B.O.M. Exporting a software bill of materials for your repository - GitHub Docs 2023. <https://docs.github.com/en/code-security/supply-chain-security/understanding-your-software-supply-chain/exporting-a-software-bill-of-materials-for-your-repository> (accessed July 7, 2025).
- [46] Qiu T., Zhu J., Chen W., Wei J. LiPSBOMaker: a prototype of multi-stage linux distribution package SBOM generator 2025:56–60. <https://doi.org/10.1145/3713081.3731738>.
- [47] H.V. Vairagade, A. Mercado, A. Ruiz Padron, A. Buniya, S. Eggers, F. Zhang, Evaluating methods of software bill of materials generation to enhance nuclear power plant cybersecurity, *Nucl. Technol.* 211 (2025) 1144–1152, <https://doi.org/10.1080/00295450.2024.2382015>.
- [48] D. Garcia, M.T. Mirakorhli, S. Dillon, K. Laporte, M. Morrison, H. Lu, et al., A landscape study of open-source tools for software bill of materials (SBOM) and supply chain security, in: *2025 IEEE/ACM 3rd International Workshop on Software Vulnerability Management (SVM)*, 2025, pp. 37–45, <https://doi.org/10.1109/SVM66695.2025.00010>.
- [49] M.F. Rabbi, A.I. Champa, M.F. Zibran, Claim vs. capability: a comparative analysis of the SBOM generation tools for rust projects, in: *Proceedings of the ACM Symposium on Applied Computing*, 2025, pp. 1712–1720, <https://doi.org/10.1145/3672608.3707940>.
- [50] L. Williams, G. Benedetti, S. Hamer, R. Paramitha, I. Rahman, M. Tamanna, et al., Research directions in software supply chain security, *ACM Trans. Softw. Eng. Methodol.* 34 (2025), <https://doi.org/10.1145/3714464>.
- [51] Huynh T.D., Moreau L. Provstore: a public provenance repository. *Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)* 2015;8628:275–7. [https://doi.org/10.1007/978-3-319-16462-5\\_32](https://doi.org/10.1007/978-3-319-16462-5_32);TAXONOMY:TAXONOMY:ACM-PUBTYPE:PAGEGROUP:STRING:PUBLICATION.
- [52] C. Omar, J. Aldrich, R.C. Gerkin, Collaborative infrastructure for test-driven scientific model validation, in: *36th International Conference on Software Engineering, ICSE Companion 2014 - Proceedings*, 2014, pp. 524–527, <https://doi.org/10.1145/2591062.2591129>.